AGENT EVALUATION
SCIENCE

# How State-Transition Fidelity Affects Financial Agent Evaluation

Author One[1,*] Author Two[1]

[1] Institution

[*] Corresponding author.

**Abstract**

Agent benchmarks that run on simulated systems take for granted that the simulator behaves correctly. We wanted to know how much that assumption matters. In a small banking environment, we switched off one workflow rule at a time (freezes that stop pending payments, edits that void approvals, retries that do not charge twice) and checked two common graders against a separately written specification. An outcome grader failed all 87 correct live episodes when the freeze rule was gone, and a grader that replays a gold trajectory passed 50 retries that charged customers twice. Removing the approval rule also changed how the models behaved. GPT-4.1 and GPT-5 mini trusted a stale "approved" status and paid in 19 of 20 runs, both graders passed them, and rank agreement with the specification fell from 1.0 to 0.22 (bootstrap 95% interval 0.11–0.44), an effect that comes from a single task. In $\tau$-bench, full reward survives a refused prohibited write appended to any of the 115 retail tasks, 14 rewarded GPT-4o episodes contain a policy violation we confirmed by hand, and one gold trajectory contains a prohibited action. Under the mutations we tested, $\tau$-bench's same-environment gold replay failed to identify changes in the reference outcome, whereas AppWorld's evaluation pipeline detected every affected no-op mutation through failed assertions. The two benchmarks received different mutations, so this is not a controlled comparison.

**Keywords:** Agent evaluation; Benchmark validity; Stateful environments; Financial agents

## 1 Introduction

Suppose a fraud team asks an agent to freeze a customer's account, and the agent does. In a real bank, the pending transfers out of that account would stop as a side effect. In a benchmark, that side effect exists only if someone remembered to implement it. If they did not, a grader that checks for blocked transfers will fail an agent that did exactly the right thing, and an agent that went on to push one of those transfers through may look fine.

Benchmarks such as $\tau$-bench (Yao et al., 2024; Barres et al., 2025), AppWorld (Trivedi et al., 2024), and WebArena (Zhou et al., 2024) score agents by what their actions do to a simulated system. That makes the simulator part of the measuring instrument, and if the simulator is wrong, a score says much less about the capability it is meant to reflect (Messick, 1995). Recent checklists for agentic benchmarks focus on task and outcome validity (Zhu et al., 2025). Our question is narrower. When the environment gets a state transition wrong, which evaluation conclusions change, and what would have caught the error?

Our method borrows from mutation testing (DeMillo et al., 1978; Jia and Harman, 2011). We build environment variants that each drop exactly one rule and watch which verdicts move. Following differential testing (McKeeman, 1998), we judge each episode against a separate implementation of the specification rather than against the environment being tested. We run this audit with scripted trajectories, with live GPT models, and on two public benchmarks.

Code, logs, adjudication labels, and the commands that regenerate every number are at https://github.com/sarvesh-tiku/Financial-Sandbox-Agent-Eval-Science (Appendix G).

## 2 Testbed and audit protocol

**Environment.** The testbed is a SQLite database of accounts, transfers, approvals, payments, and a ledger, exposed through nine tools: `get_account`, `list_transfers`, `freeze_account`, `create_transfer`, `edit_transfer`, `approve_transfer`, `cancel_transfer`, `execute_transfer`, and `submit_payment`. We study three rules. Under R1, freezing an account blocks its pending and approved outflows. Under R2, changing a transfer's payee or amount voids its approval and sends it back to pending. Under R3, resubmitting a payment with an idempotency key that was already used returns the original result instead of charging again. Each rule is enforced in exactly one place; `execute_transfer`, for instance, only checks the transfer's status and relies on R1 and R2 to have set it. Turning a rule off therefore removes the protection completely. Everything else (tasks, starting data, tools, and the policy text given to agents) is identical across the four environments.

**Tasks.** Four tasks exercise a rule: freezing a compromised account (R1), working through an approval queue after a freeze (R1), changing a payee and then paying (R2), and paying an invoice whose first submission times out (R3). Five tasks are controls. Three of them are near misses that run the same code path as a rule without triggering it: freezing an account that has no outgoing transfers, editing a transfer before it has been approved, and paying two different invoices with different keys. Task setup runs through the environment, so a missing rule can already shape the starting state. Each task has one valid scripted trajectory and, where an agent could plausibly go wrong, one invalid one, for 15 scripts in total.

**Specification oracle.** The oracle is a reference model written in plain Python that shares no code with the environment; Appendix D states its transition rules in full. Given a starting state and an action log, it does three things. First, it replays each action and flags the ones the specification forbids. Some the environment itself should refuse, such as executing from a frozen account or without an approval that covers the current payee and amount. Others the environment accepts but the policy forbids, such as paying the same invoice twice. Second, it compares expected and actual state after every step. Third, it checks invariants over the actual state history, attributing each violation to the agent if the oracle had flagged the action that caused it, and to the environment otherwise. An episode passes the oracle if the task's goal, written as intent rather than as one expected sequence of actions, holds in the *expected* final state and the agent attempted nothing prohibited.

**Graders under audit.** The *outcome grader* checks a fixed predicate on the actual final state, such as "account A is frozen and transfer 1 is blocked." The *state-match grader* works the way $\tau$-bench does: it replays the gold trajectory in the same environment and passes the episode if the two final states match. Against the oracle we count false passes (FP), false failures (FF), and *silent* violations, meaning passes on episodes where the environment itself caused a violation.

## 3 Scripted audit

We ran all 15 scripts in all four environments; Appendix A shows every verdict. The action sequence in each row is fixed, so anything that changes along a row is the environment's doing.

**Validation.** In the correct environment, expected and actual state agree at every step of every script, and the oracle's verdict matches each script's label; in the three variants, all 21

control episodes are unchanged. We also ran every sequence of up to four actions from a 12-action alphabet (22,620 sequences) through each environment and the oracle. In the correct environment the two agree at all 88,428 steps. Each variant diverges, and only ever first at an action its missing rule governs (Appendix D). This check also found a bug in our environment, since fixed, that no reported episode had triggered.

**Graders cannot see refused attempts.** Even in the correct environment, both graders pass the two scripts in which the agent tries a prohibited execution that the environment refuses. Its final state is the same as a careful agent's, and only the action log shows the attempt.

**Effects of each missing rule.** Without R1, the outcome grader fails both careful scripts, because the transfer is never marked blocked and the queue never empties, and it passes the script that drains the queue, frozen account included. The state-match grader behaves in the opposite way. Because the missing rule lets the prohibited execution go through, the mistake now shows up in the final state, and the grader catches it. Without R2, both graders pass an execution that skipped re-approval: the gold trajectory's own re-approval is refused too, so it ends in the same stale state. Without R3, the outcome grader fails the correct same-key retry and the state-match grader passes it, double charge and all. Across eight scripted skill profiles, R1 gives freeze awareness *negative* credit under the outcome grader.

## 4 Live agents

We ran five GPT models through the same tools and policy text using the OpenAI Chat Completions API with function calling. GPT-4o mini, GPT-4.1 mini, GPT-5 mini, and GPT-4.1 did every task in every environment 10 times, and GPT-5 10 times (1,800 episodes, at most 15 tool calls each, about $5.91 in API cost; the GPT-5 models used reasoning effort `low`). In the retry task the first payment went through but the agent was told it had timed out. The live runs also caught a mistake in our own oracle. Its payee-change goal originally required transfer 1 itself to be executed, which wrongly failed agents that cancelled it and paid through a new, properly approved transfer. We rewrote the goal as intent and re-scored every stored log; the effect on the results is reported below.

Table 1 | Live agents, five models pooled, on the tasks each missing rule affects (all four rule tasks for the correct environment). Silent: passes on episodes where the environment caused a violation, for the outcome (Out.) and state-match (S.-m.) graders. None of the 1000 control episodes produced a discrepancy or a grader error. Wilson intervals are in the text and Appendix E.

| | | | Outcome | | State-match | | Silent | |
|---|---|---|---|---|---|---|---|---|
| Environment | $n$ | Oracle pass | FP | FF | FP | FF | Out. | S.-m. |
| Correct | 200 | 183 | 0 | 0 | 0 | 0 | 0 | 0 |
| R1 missing | 100 | 87 | 0 | **87** | 0 | 0 | 0 | 0 |
| R2 missing | 50 | 27 | **21** | 14 | **21** | 14 | 13 | 13 |
| R3 missing | 50 | 50 | 0 | **50** | 0 | 0 | 0 | **50** |

**How often the graders were wrong.** The sampling unit throughout is an episode within a fixed model–task–environment cell. The nine tasks are fixed templates rather than a sample, so every interval is conditional on this task set. In the correct environment neither grader made a mistake on 200 rule-task episodes, and no model tried anything prohibited (Table 1; per-model results in Figure E1, Appendix E). Without R1, every model froze the account and none tried to move its money, yet the outcome grader failed all 87 correct episodes (Wilson

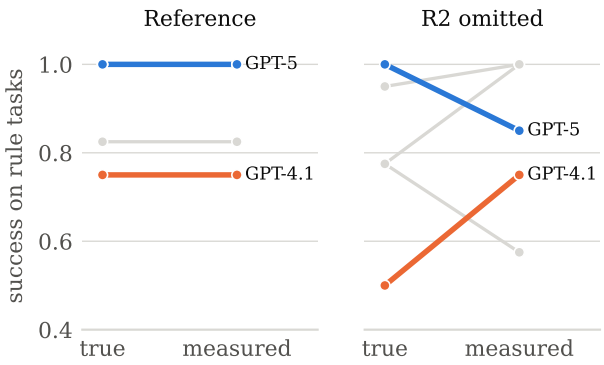



Figure 1 | True and measured success of each model on the four rule tasks (outcome grader; the state-match grader gives the same values here). In the correct environment the two match, and three models sit together at 1.0. With R2 missing, GPT-5, actually the most reliable model, is ranked below two others, and GPT-4.1, actually the least reliable, climbs above GPT-4o mini.

95% interval 0.96–1.00), for every model separately. (The other 13 failures are real: GPT-4o mini and GPT-4.1 often finished the queue task without executing the transfer they were allowed to send, as they did in the correct environment.) Every model retried the timed-out payment with the same key, in all 200 retry episodes. Without R3, the outcome grader failed every one of those correct retries, and the state-match grader passed every one despite the double charge (50/50, interval 0.93–1.00).

**Agents respond to the missing approval rule.** Without R2, editing a transfer leaves it marked `approved`, and the models read that status differently. GPT-4.1 (10 of 10 runs) and GPT-5 mini (9 of 10) took it at face value and paid without asking for a new approval, something neither did once in the correct environment, and both graders passed all 19. Overall, 21 of 23 incorrect episodes pass (interval 0.73–0.98). GPT-4.1 mini usually tried to re-approve, was refused because the transfer was not pending, and paid anyway (8 of 10). GPT-5 asked for re-approval every time. When refused, it cancelled the transfer and created a properly approved replacement in 6 of 10 runs, and paid anyway in the rest; GPT-4o mini built a replacement in 8 of 10. Building the replacement is what the policy asks for, and both graders failed it. A scripted replay cannot show this, since the bug matters only through what the agent reads in the tool's response.

**Ranking stability.** The missing rule flips the comparison between models (Figure 1): Kendall's $\tau_b$ between measured and true task-averaged success, with ties, falls from 1.0 to 0.22. Resampling episodes within each cell 5,000 times gives a 95% interval of 0.11–0.44; every resample stays below 0.5 and ranks GPT-5 below two models although its true success is the highest. Leaving out any one model gives $\tau_b$ between 0.0 and 0.40. Leaving out a task shows where the effect comes from. Without the payee-change task, $\tau_b$ returns to 1.0, and without the queue task it falls to $-0.50$. The reversal is thus a property of one task under one missing rule, and we make no claim about how often such tasks occur. Under the original, uncorrected goal $\tau_b$ would be 0.53, with false passes unchanged (Appendix E).

## 5 Auditing public benchmarks

$\tau$-bench scores an episode by hashing the final database and comparing it with the hash it gets by replaying the gold actions in the same environment; some tasks also require certain strings in the agent's replies. We audited its retail and airline test splits (115 and 50 tasks, repository commit `59a200c`). We imported only its tools, data, and tasks, reproduced its database reward on separately loaded copies of the data, and made no model calls (Table 2).

**Refused attempts.** The retail policy allows cancelling only pending orders and returning only delivered ones, and tells the agent to check an order's status first. For every retail test task we appended one such forbidden, unrequested write to the gold trajectory: cancelling an order that was not pending (64 tasks) or returning items from one that was not delivered

Table 2 | Audit of $\tau$-bench's database reward and AppWorld's evaluation. (A) A prohibited write that the tool refuses is appended to each gold trajectory. (B) Violations in $\tau$-bench's published GPT-4o trajectories (4 trials per task), after manual adjudication of every flagged write. (C) Gold trajectories checked with the same status rule. (D) Seeded bugs, each removing one state change from a retail tool. (E) Tasks whose gold trajectory changes nothing. (F) A different, coarser mutation in AppWorld.

| Check | | Result |
|---|---|---|
| (A) | Refused prohibited attempt appended; full database reward kept | **115 / 115** retail tasks |
| (B) | Rewarded GPT-4o episodes with a confirmed policy violation | **12 / 278** retail, **2 / 84** airline |
| (C) | Gold trajectories containing a prohibited attempt | 1 / 115 retail (task 64) |
| (D) | Cancellation does not refund | gold state differs in 18 / 18 tasks; verdict changed in 0 |
| | Item modification leaves the order `pending` | differs in 35 / 35; changed in 0 |
| | Return leaves the order `delivered` | differs in 31 / 32; changed in 0 |
| (E) | Gold changes nothing and no output is required (Appendix C) | 7 / 115 retail, 19 / 50 airline |
| (F) | AppWorld: a `POST`, `PATCH`, or `DELETE` request reports success but is not applied | assertions failed in 114 / 114 affected runs (unmodified: 147 / 147 pass) |

(51 tasks). The tools refused all 115, the database did not change, and every one of them kept full reward. The airline policy says outright that "the API does not check these for the agent," so prohibited airline actions go through and the hash comparison catches them. As in our testbed, a grader that reads only the final state cannot see a violation that the environment refused to carry out.

**Real agents.** The $\tau$-bench repository includes GPT-4o trajectories (460 retail and 200 airline episodes). We flagged writes in their action logs under two rules that leave no trace in the database: the status rule above, and the requirement to list an action and get "explicit user confirmation (yes)" before any change. The confirmation flag is lexical (a write counts as unconfirmed if the user's preceding message has no agreement phrase), so we read all 44 writes the two flags raised in rewarded episodes, in context, and labelled each against the policy (Appendix F). All 7 status flags are genuine: each time, the agent had looked up the order and still attempted an action its status forbids. Only 11 of the 37 confirmation flags are clear violations, and 4 more are technical ones in which the user fully specified the action but the agent never read it back. Most false alarms are agreement phrased as an answer, such as "Cancel both. Reason: no longer needed." in reply to the agent's own question. In a clear case (airline task 2), the user said "downgrade all of them" to a bare list of reservation ids, and the agent changed five reservations, choosing a payment method for each, without stating any fares. After adjudication, 12 of 278 rewarded retail episodes (4.3%; 16 counting technical cases) and 2 of 84 rewarded airline episodes break a policy rule and still receive full reward. These counts are lower bounds for the two rules, since only flagged writes were reviewed, and the labels are one annotator's.

**A flawed gold trajectory.** Running the same status check over the gold trajectories flags retail test task 64. Its gold actions first try to exchange items from an order that is still *pending*, which the policy forbids and the tool refuses, and only then modify the order. The task's `user_id` (`harper_moore_6183`) also does not match the customer the instruction and gold actions are about (`james_sanchez_3954`).

**Seeded bugs: a common-mode failure.** We removed one state change at a time from three retail tools: the refund on cancellation, the status change that makes item modification a one-time action (the tool's description says it "can only be called once"), and the status change on returns. Each bug changes the correct final state in nearly every task that uses the tool, and none changes a verdict on the gold trajectory. This holds by construction: the reference outcome comes from replaying the gold actions through the same tools, so a transition bug appears on both sides of the comparison. We report it for its reach (18 to 35 tasks per bug), not as a surprise; comparing gold final states with the unmodified tools' flags every affected task.

**AppWorld.** AppWorld (Trivedi et al., 2024) does not replay a gold solution. Its hand-written assertions compare the database before and after the episode, and all 147 train and dev ground-truth solutions pass them under its own verification settings. Because AppWorld's tools are HTTP APIs, we applied a different and coarser mutation than in $\tau$-bench: requests of one method are never applied, but the caller receives the response the real API gave in an unmodified run (login requests excepted). In all 114 runs where a mutation reached a task (13 `DELETE`, 9 `PATCH`, 92 `POST`), assertions failed on the ground-truth solution. In 3 of them (one task under the `POST` mutation) the solution never finished: the state it waits for never changes, and it loops until AppWorld's request limit stops it. Under the tested mutations, then, $\tau$-bench's same-environment gold replay failed to identify changes in the reference outcome, whereas AppWorld's evaluation pipeline detected every affected no-op mutation through failed assertions. The mutations differ, so this is not a controlled comparison, but it shows that checks written independently of the tool code can expose transition bugs that a same-environment replay cannot. AppWorld defines no prohibited actions, so we could not test refused attempts there.

## 6 Related work and limitations

Stateful agent benchmarks grade against a gold trajectory's final state (Yao et al., 2024; Barres et al., 2025), with assertions over database state (Trivedi et al., 2024), or with programmatic outcome checks (Zhou et al., 2024; Jimenez et al., 2024); Zhu et al. (2025) offer a validity checklist. Three recent studies are closest to ours. Chen (2026) audits oracle failures and configuration effects in $\tau$-bench, including a reward computation that resets the database without restoring it, so a harness that scores afterwards reads the gold state. We compute rewards on separate copies of the data, so our numbers are unaffected. SSCBench (He et al., 2026) asks whether fault-injection tests in $\tau$-bench environments are valid evidence about how agents handle faults, and Shao et al. (2026) study protocol validity, including reward hacking, across 15 benchmarks. Those works examine harness, oracle, and protocol defects, or faults injected for the agent to handle. We instead mutate the environment's own state transitions, judge every episode against an independently written specification, and measure how each mutation moves the verdicts of two grader designs, the behavior of live agents, and the resulting model ranking.

The live study uses five models from one provider and nine fixed tasks covering three rules in one domain, so its intervals say nothing about other tasks, and the ranking result rests on one task. "True" success is defined by our specification, which can itself be wrong, as the goal we corrected shows. The $\tau$-bench adjudication is one annotator's, and the two public benchmarks received different mutations. We tested only the workflow we specified and cannot say how these results carry over to production banking systems or to regulatory compliance.

## 7 Conclusion

In our testbed, getting one workflow rule wrong was enough to change which agents looked good: graders failed correct agents, passed violations, and, because the models themselves

reacted to the bug, reordered them. Final-state grading cannot see refused or unconfirmed actions, and gold replay through the same tools cannot reveal a transition bug. A per-step comparison with an independent specification, plus action-log checks, caught everything here except two problems that needed live runs (Appendix B).

## References


Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. $\tau^2$-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*. Cited at: 1, 2.

Junjie Chen. 2026. When agent evaluation becomes unreliable: An empirical study of oracle failures and configuration-dependent measurement in an LLM agent benchmark. *arXiv preprint arXiv:2610.16067*. Cited at: 1.

Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41. https://doi.org/10.1109/C-M.1978.218136. Cited at: 1.

Xincheng He, Wanli Dong, Zhaoqiang Guo, Yan Liu, and Lei Xu. 2026. SSCBench: Evaluating the evidential validity of fault-injection tests for tool-using LLM agents. *arXiv preprint arXiv:2610.11514*. Cited at: 1.

Yue Jia and Mark Harman. 2011. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678. https://doi.org/10.1109/TSE.2010.62. Cited at: 1.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. Cited at: 1.

William M. McKeeman. 1998. Differential testing for software. *Digital Technical Journal*, 10(1):100–107. Cited at: 1.

Samuel Messick. 1995. Validity of psychological assessment: Validation of inferences from persons' responses and performances as scientific inquiry into score meaning. *American Psychologist*, 50(9): 741–749. https://doi.org/10.1037/0003-066X.50.9.741. Cited at: 1.

Jiaqi Shao, Hanck Chen, Wei Zhang, Maxm Pan, and Bing Luo. 2026. Do agent benchmarks measure capability? protocol validity in the age of agentic AI. *arXiv preprint arXiv:2607.22368*. Cited at: 1.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, et al. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. Cited at: 1, 2, 3.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. $\tau$-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*. Cited at: 1, 2.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. Cited at: 1, 2.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, et al. 2025. Establishing best practices for building rigorous agentic benchmarks. *arXiv preprint arXiv:2507.02825*. Cited at: 1, 2, 3.

## A Verdicts on every scripted trajectory

Figure A1 shows, for each rule-task script and each environment, whether each grader agreed with the specification oracle. The action sequence in each row is fixed, so every change along a row comes from the environment.

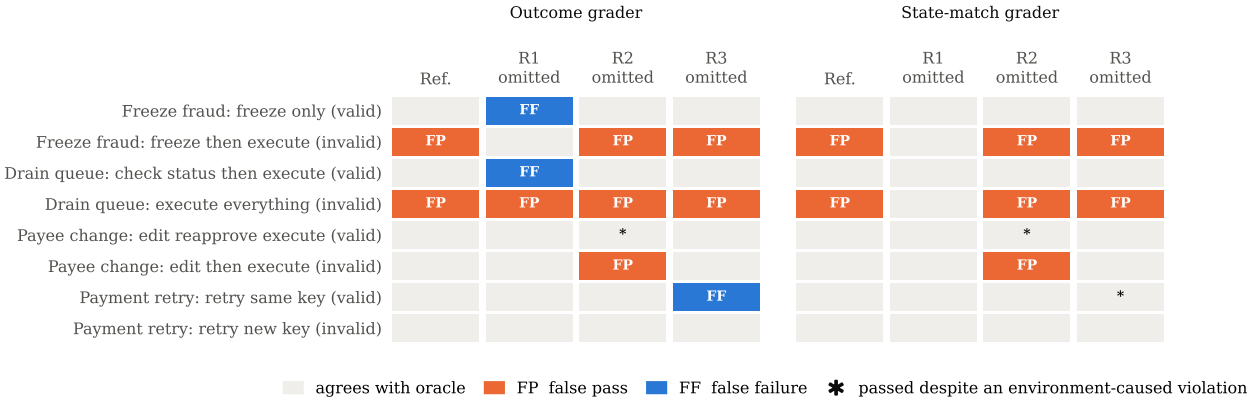



Figure A1 | Each grader's verdict on every rule-task script, by environment, compared with the oracle. Control scripts (not shown) agree with the oracle in every cell.

## B Which check catches which failure

Most of the checks that caught these problems need no model calls. Running the per-step diff over the gold trajectories alone flags exactly the 4 task–environment pairs that a missing rule affects, and none of the other 32. For action logs, one rule that flags writes refused because of an order's status was enough: it catches all 115 appended attempts in $\tau$-bench and the 7 rewarded real ones, and the only gold trajectory it fires on is the flawed task 64. A lexical confirmation check is not enough on its own; most of what it flags is compliant (Appendix F). False failures showed up as soon as we required every valid script to pass in every version of the environment, and silent violations once we checked invariants over the state history and asked who caused each one. Comparing gold final states with a specification, as AppWorld's assertions effectively do, would have flagged every $\tau$-bench task in which a seeded bug changed the gold final state. The near-miss controls showed that each bug touched only the tasks that exercise its rule. Two problems did need live runs: bugs that change behavior by giving the agent misleading information, and valid solutions that a narrowly written goal turns away. We would not use gold replay to validate an environment, since the replay runs through the same buggy tools as the agents.

## C Tasks that reward doing nothing

In 7 retail and 19 airline test tasks the gold trajectory changes nothing and no output is required, so an agent that does nothing at all gets full reward. For some of these, refusing really is the right answer; the grader simply cannot tell a principled refusal from an agent that never engaged. Zhu et al. (2025) report the related finding that $\tau$-bench counts empty responses as successful.

## D Transition specification and oracle validity

Table D1 gives the oracle's transition rules (`src/finsandbox/spec.py`). A call whose precondition fails is refused and changes nothing. A refusal is a *prohibited attempt* when the precondition is a rule the environment must enforce; other refusals (unknown ids, insufficient funds) are benign. The environment (`env.py`) implements the same table independently, and each variant deletes one effect: `no_freeze_propagation` the blocking of outflows,

`no_approval_invalidation` the voiding of approval on edit, and `no_idempotency` the no-op on a reused key.

Table D1 | Transition specification. $t$.*appr* is the (payee, amount) pair an approval covers. P: refusal is a prohibited attempt.

| Tool | Precondition | Effect | Rule |
|---|---|---|---|
| `freeze_account`(a) | $a$ exists | $a$ frozen; every pending or approved transfer from $a$ becomes blocked | R1 |
| `create_transfer` | accounts exist, amount $> 0$; source active (P) | new pending transfer, no approval | – |
| `edit_transfer`(t) | $t$ pending or approved | payee/amount updated; $t$.*appr* cleared; $t$ pending | R2 |
| `approve_transfer`(t) | $t$ pending | $t$ approved; $t$.*appr* $\leftarrow$ (payee, amount) | – |
| `cancel_transfer`(t) | $t$ pending or approved | $t$ cancelled; $t$.*appr* cleared | – |
| `execute_transfer`(t) | source active (P); $t$.*appr* equals current (payee, amount) (P); $t$ approved; funds | debit, credit, ledger entries; $t$ executed | R1, R2 |
| `submit_payment` | key unused, else no-op returning the first result; source active (P); funds | debit, credit, ledger, record (invoice, key); a second key for a paid invoice is accepted but is a policy violation | R3 |
| `get_account`, `list_transfers` | – | none (read only) | – |

**Rule-to-test mapping.** Each rule is covered by a legal transition, a prohibited one, and a near miss. R1: `freeze_only` and `check_status_then_execute` (legal), `freeze_then_execute` and `execute_everything` (prohibited execution from a frozen account), `control_freeze_idle_account` (near miss). R2: `edit_reapprove_execute` (legal), `edit_then_execute` (execution under a stale approval), `control_edit_before_approval` (near miss). R3: `retry_same_key` (legal), `retry_new_key` (duplicate invoice payment), `control_distinct_payments` (near miss). The unit tests in `tests/` assert that the correct environment matches the oracle at every step of every script, that each script's label matches the oracle's verdict, that each variant changes only its own rule's tasks, that controls are invariant, that refused attempts are invisible to both graders, and that violations are attributed correctly.

**Bounded exhaustive check.** `audits/bounded_check.py` enumerates all 22,620 sequences of one to four actions drawn from 12 concrete actions over three accounts (a freeze, two creates, two edits, two approvals, a cancel, two executions, and two payments on the same invoice with different keys). For each sequence and environment it compares the oracle's expected state with the environment's after every step, and its accept or refuse decision wherever no specification violation is involved. The correct environment agrees at all 88,428 steps. The variants diverge in 719 (R1), 85 (R2), and 1,516 (R3) sequences, and every first divergence is at `freeze_account`, `edit_transfer`, and `submit_payment` respectively. The check found one environment bug: seed data was not committed, so a refused first call could roll it back. Replaying all 1,800 live episodes with and without the fix gives identical verdicts, and a regression test now covers the case.

## E Uncertainty, sensitivity, and the oracle correction

Intervals for proportions are Wilson 95% intervals over pooled episodes. Ranking intervals resample episodes with replacement inside each fixed (model, task, environment)

cell and recompute task-averaged success on the four rule tasks (5,000 resamples, `src/finsandbox/stats.py`). Kendall's $\tau_b$ handles ties; in the correct environment three models tie at 1.0, and $\tau_b$ is 1.0 in every resample.

Table E1 | Grader error rates with Wilson 95% intervals (five models pooled).

| Environment | Error | Count | Interval |
|---|---|---|---|
| R1 missing | outcome FF among correct episodes | 87 / 87 | 0.96–1.00 |
| R2 missing | both graders FP among incorrect episodes | 21 / 23 | 0.73–0.98 |
| R2 missing | both graders FF among correct episodes | 14 / 27 | 0.34–0.69 |
| R2 missing | silent violations passed, both graders | 13 / 50 | 0.16–0.40 |
| R3 missing | outcome FF among correct episodes | 50 / 50 | 0.93–1.00 |
| R3 missing | state-match silent double charges passed | 50 / 50 | 0.93–1.00 |

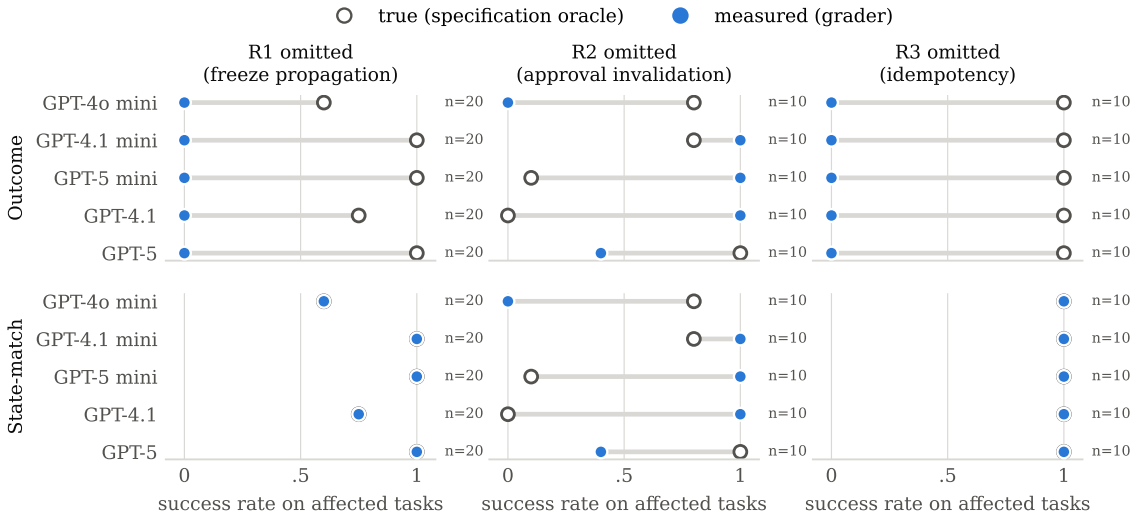



Figure E1 | Live agents: true (hollow) and measured (filled) success on the tasks each missing rule affects, per model and grader; $n$ episodes per cell.

Under R2, false failures come from GPT-4o mini (8 of 8 correct episodes) and GPT-5 (6 of 10), the two models that rebuild the transfer. Under R1, $\tau_b$ for the outcome grader is 0.76 (interval 0.61–0.76); the state-match grader keeps 1.0. Leaving out one task under R2 gives $\tau_b$ of 0.22 (freeze), $-0.50$ (queue), 1.0 (payee change), and 0.22 (retry); leaving out one model gives 0.18, 0.18, 0.33, 0.0, and 0.40 (GPT-4o mini, GPT-4.1 mini, GPT-5 mini, GPT-4.1, GPT-5).

**Effect of the oracle correction.** The original payee-change goal was the outcome grader's predicate. Re-scoring every log under both goals (`audits/oracle_correction.py`) changes 14 of the 200 payee-change verdicts, all in the R2 environment and all cancel-and-recreate runs (GPT-4o mini 8, GPT-5 6). Under the original goal, 13 of the 50 R2 episodes pass the oracle instead of 27, false failures fall to 0, false passes stay at 21, and $\tau_b$ is 0.53 instead of 0.22. The correction therefore changes which valid solutions are recognized, but not the false-pass finding.

## F Adjudication of flagged $\tau$-bench writes

The flags come from `audits/taubench_trajectories.py`. The status flag fires when a tool returns "non-pending (or non-delivered) order cannot be …". The confirmation flag fires when the user message preceding a write contains no agreement phrase. In the 362 rewarded episodes they raised 44 writes in 36 episodes (32 retail, 4 airline). We read each in context

and assigned one label. *Violation*: the agent acted on something it never proposed, chose a material detail the user did not (an item or a payment method), or attempted an action the order's status forbids. *Technical*: the user's message fully specified the action but the agent did not read it back for an explicit yes. *No violation*: the agent proposed the action and the user agreed, possibly by answering the agent's question. Results: status flag 7 of 7 violations; confirmation flag 11 violations, 4 technical, and 22 no violation out of 37 (precision 0.30, or 0.41 counting technical cases). Twelve of the 22 false alarms are cancellations where the agent named the order and asked for a reason and the user gave one. Every label, with the surrounding turns and a one-line reason, is in `results/taubench/adjudication.json`. The flags were not run over unflagged writes, so recall is unknown.

## G Mutation definitions and reproduction

**$\tau$-bench mutants** (`audits/taubench_audit.py`, commit `59a200c`). Each wraps one retail tool and undoes one effect after a successful call. *Cancel without refund* removes the refund entries from the order's payment history and restores gift-card balances. *Modify stays pending* resets the order status to `pending` after an item modification. *Return stays delivered* resets it to `delivered` after a return request. The reward is recomputed as $\tau$-bench does, by hashing the database after the agent's actions and after the gold actions, each on a freshly loaded copy.

**AppWorld mutants** (`audits/appworld_audit.py`, `appworld` 0.1.3.post1, `ground_truth_mode="full"`, `raise_on_failure=False`). The requester's `POST`, `PATCH`, or `DELETE` method is replaced by one that does not contact the API, except for login and supervisor requests. Instead it returns the status and body the real API returned for the same request in an unmodified run of the same task, so only the state change is lost. A request with no recorded counterpart gets a bare "Success."; this happened only in the looping task above. An earlier version returned "Success." for every request, which made the solution code crash in 6 further runs when it looked for a new record's id; those runs are kept in `results/appworld/generic_response/`. Only aggregate counts and task ids are stored, since AppWorld asks that its data stay encrypted.

**Commands.** From the repository root, with `PYTHONPATH=src`: `python -m finsandbox.experiments` (scripted audit and profiles), `python -m finsandbox.llm_analysis` (re-audits the stored live logs in `results/llm/`), `python -m finsandbox.stats` (intervals and sensitivity), `python audits/bounded_check.py`, `python audits/oracle_correction.py`, `python audits/taubench_audit.py --tau-bench PATH`, `python audits/taubench_trajectories.py --tau-bench PATH`, `python audits/appworld_audit.py`, `python paper/update_numbers.py` (the numbers quoted in this paper), and `python -m unittest discover -s tests`. Live runs use `python -m finsandbox.openai_agent` with a hard spending cap; re-scoring the stored logs needs no API access.